

Bitácora de desarrollo del Backend

API de Productos, Insumos y Proveedores

Información general

Proyecto: API de consulta de Productos, Insumos y Proveedores

Tecnología: Python + FastAPI

Asistente de IA: Claude Code (Anthropic)

Registro de prompts

Prompt 1: Diseño y arquitectura inicial de la API

Herramienta: Claude Code

Prompt (versión redactada):

Desarrolla una API REST utilizando FastAPI que exponga tres endpoints de consulta: Productos, Insumos y Proveedores. Los tres endpoints deben retornar el listado de entidades correspondiente a cada recurso.

La información debe gestionarse bajo una arquitectura basada en grafos dirigidos, compuesta por nodos y aristas. Cada nodo representa una entidad del sistema y debe contener un identificador, un nombre, la capa a la que pertenece (producto, insumo o proveedor) y, opcionalmente, una categoría. Las aristas representan relaciones de dependencia del tipo REQUIERE, donde el nodo de origen corresponde a la entidad que necesita y el nodo de destino a la entidad requerida.

Respetar estrictamente la estructura de nodos y aristas definida, así como la forma en que se relacionan entre sí. Para esta etapa, los datos deben manejarse como un conjunto simulado (mock) almacenado en memoria dentro de la propia API, sin conexión a base de datos. Únicamente se permite modificar los datos de ejemplo, manteniendo intacto el esquema.

Objetivo: Establecer la base del backend con una arquitectura clara y escalable, que permita validar el modelo de datos antes de integrar una fuente de datos real.

Requerimientos de diseño solicitados:

- **Framework:** FastAPI, aprovechando su validación de datos y su documentación automática.

- **Endpoints:** tres endpoints GET con una lógica equivalente, uno por cada tipo de entidad.
- **Arquitectura basada en grafos:** **nodos**, que representan productos, insumos y proveedores, y **aristas**, que representan relaciones de dependencia del tipo REQUIERE.
- **Fidelidad estructural:** el esquema de nodos y aristas y sus conexiones debían mantenerse sin cambios.
- **Persistencia:** datos simulados en memoria para esta primera fase.

Resultado obtenido: Claude Code generó una estructura modular de tres archivos:

- `models.py` con los modelos de datos.
- `data.py` con el grafo de ejemplo.
- `main.py` con la aplicación y los endpoints.

Explicación del código

`models.py`: los moldes de los datos

Este archivo define cómo tiene que verse cada dato, usando Pydantic, una librería que valida automáticamente que la información cumpla la forma esperada.

- Capa solo permite tres valores: producto, insumo o proveedor. Si llega otro valor, se rechaza.
- Nodo es cualquier elemento del sistema, como un producto, un insumo o un proveedor. Tiene id, nombre, capa y, de forma opcional, categoría.
- Arista es la flecha que une dos nodos y dice "este necesita a este otro". Por ejemplo, un producto requiere un insumo, y ese insumo requiere un proveedor.
- Grafo es el paquete completo, con la lista de todos los nodos y la lista de todas las conexiones.

`data.py`: los datos quemados

Aquí vive `MOCK_GRAFO`, el grafo de ejemplo con los productos, insumos y proveedores y sus conexiones. Funciona como una base de datos de prueba mientras el proyecto crece.

`main.py`: la API en sí

Aquí se crea la aplicación y se definen las rutas.

- `/` es una ruta de prueba que solo responde "API funcionando", para confirmar que el servidor está arriba.
- `/productos`, `/insumos` y `/proveedores` hacen lo mismo. Recorren todos los nodos del grafo y devuelven solo los de la capa correspondiente. Es como tener una lista mezclada y filtrarla por tipo.

- `response_model=List[Nodo]` le dice a FastAPI que la respuesta será una lista de nodos. Así valida la salida y la muestra bien documentada.